

## 1. 실제 실행 구조 - VM / Docker / 프로세스 경계

### EMS VM

**OPC UA Simulator**  
opcua\_simulator.py  
State / Good / Reject 생성  
TCP 4840

OPC UA  
:4840  
→

### OpenMES Host - WSL2 / Docker

#### DOCKER

**OPC UA Gateway**  
Signal 구독 + Heartbeat 전송

**OpenMES Stack**  
Backend / API / DB  
Work Order / Machine / Runtime API

REST API  
HTTP  
→

### AWSQUICK-LAB VM

**Shop-Floor App**  
app.py  
Work Order 조회 / Serial / PASS-FAIL / CSV

**Mock Printer**  
mock\_printer.py  
TCP 9100 Label 수신

## 2. 정상 프로세스 - 먼저 정상 데이터 흐름을 확인한 뒤 장애를 분리

**1 설비 데이터 생성**  
EMS Simulator가 State / Good / Reject 값을 생성하고 OPC UA로 제공

**2 Gateway 수집**  
Docker Gateway가 Signal을 구독하고 OpenMES로 Reading + Heartbeat 전달

**3 MES 정보 제공**  
OpenMES가 Work Order, Machine Status, Runtime 상태를 API로 제공

**4 현장 처리**  
Shop-Floor App에서 Serial → PASS/FAIL → CSV 저장 → Label/TCP 9100 출력

## 3. 장애 3종 검증 - 서로 다른 Fault Domain을 구분

### FAULT 01 - DEVICE CONNECTIVITY

#### OPC UA Endpoint / 설비 연결 장애

**증상**  
Gateway가 OPC UA endpoint에 연결하지 못하고 재시도. 설비 데이터가 OpenMES까지 전달되지 않음.

```
old endpoint:  
opc.tcp://192.168.40.128:4840  
  
retry ...  
connection failed
```

**확인**  
Simulator 실행 상태 → IP → TCP 4840 → OpenMES Connection 설정 순서로 확인.

**조치**  
Simulator를 EMS VM으로 정리하고 endpoint를 192.168.40.132:4840으로 수정.

```
CHANGED GoodCount: 73  
CHANGED GoodCount: 74  
CHANGED GoodCount: 75
```

검증: Gateway가 실제 OPC UA 값을 다시 구독하고 카운터 변화 수신

### FAULT 02 - MIDDLEWARE / GATEWAY

#### Gateway 중단 / Heartbeat Lost

**시험**  
OPC UA Gateway를 의도적으로 중단하고 OpenMES의 runtime heartbeat 기준으로 장애 감지.

```
docker stop openmes-opcua-gateway  
  
Communication: HEARTBEAT LOST  
State: UNKNOWN  
Good: 0 (last known)  
Reject: 0 (last known)
```

**판정**  
20초 이상 heartbeat 미수신 시 값을 "현재값"으로 오인하지 않고 UNKNOWN + last known으로 표시.

**복구**  
Gateway 재시작 후 heartbeat 재개.

```
Communication: HEALTHY  
State: RUNNING
```

검증: Normal → Fault → Detection → Recovery 전체 사이클 확인

### FAULT 03 - PRINTER / TCP

#### TCP Printer 연결 장애

**시험**  
Mock Printer를 실행하지 않은 상태에서 제품 PASS 처리와 Label 출력을 수행.

```
Serial: SN-005  
Result: PASS
```

```
Production record saved to CSV.  
Printer simulation file generated: label.zpl  
ERROR: Printer connection refused.
```

**분리**  
MES 생산 처리와 Printer 통신을 별도 fault domain으로 판단.

**결과**  
Work Order/Serial/PASS 기록과 CSV 저장은 정상 완료, TCP 9100 출력만 실패.

검증: "MES 장애"가 아니라 "Printer 통신 장애"로 정확히 격리

### 핵심 역량

본 실습은 작업지시 조회, 설비 상태 확인, 시리얼 스캔, PASS/FAIL 기록, 생산 데이터 저장 및 라벨 출력까지 일반적인 제조 현장의 생산 프로세스를 시뮬레이션하였습니다. 또한 Device → Gateway → MES → Shop-Floor Terminal → Printer로 이어지는 각 구간에 의도적인 장애 상황을 구성하여, 정상 상태와 고장 상태를 비교하고 장애 발생 위치를 구분한 뒤 복구까지 검증할 수 있도록 실무적으로 구성하였습니다. 이를 통해 기존 통신·네트워크 경험을 제조 현장 IT/OT 시스템의 연결성 점검, 장애 분리 및 복구 확인 방식으로 확장하였습니다.

### Technology

Linux · Docker · OpenMES · REST API · Laravel Sanctum · OPC UA · Python · TCP/IP

### Artifacts

app.py · mock\_printer.py · production\_log.csv · label.zpl · README.md